CXXVIII - TP 1 Pyxel, qu'est-ce que c'est ?

Pyxel est un moteur de jeu rétro en Python. Il vise à simplifier la création de jeux en pixel art, avec des contraintes graphiques et sonores rappelant les consoles 8-bit.
Pour savoir tout ce qu'il est possible de faire avec cette librairie, ainsi que des exemples de projets pour lesquels elle a été utilisée, on pourra voir les pages web Pyxel (documentation), ou Pyxel studio (exemples).
Cette librairie python met également à disposition un petit programme, intitulé "Éditeur Pyxel" qui permet de faciliter :

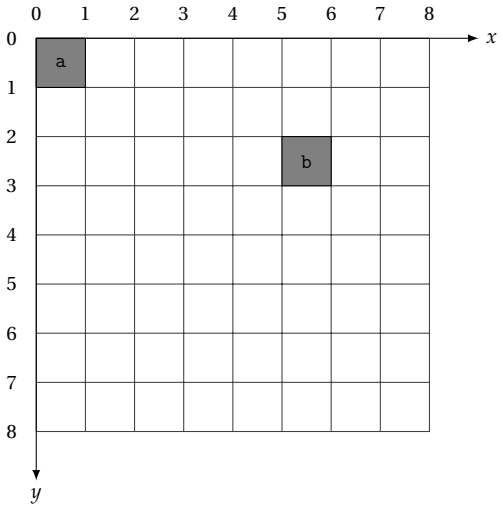
- la création et la modification des **images** (sprites, tilesets).
- la création des **tilemaps** (agencement des tuiles pour les niveaux).
- la composition des **effets sonores (SFX)**.
- l'arragement des **musiques** (en utilisant les SFX).
- la gestion les ressources du jeu (fichiers `.pyxres`).

Cela fait beaucoup de nouveaux mots ! Nous allons dans ces TPs essayer de comprendre ce que tout cela signifie, ainsi que comment utiliser les fonctions principales de la bibliothèques.

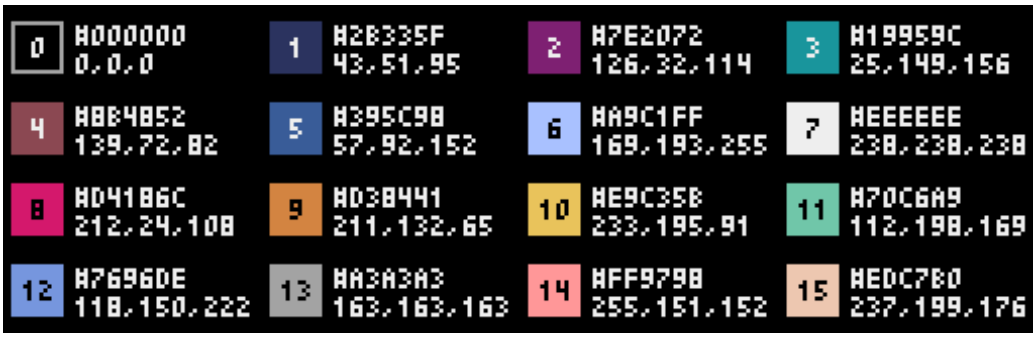
1 Mon premier programme avec Pyxel

On considère que la fenêtre affichée à l'écran comme une image : il s'agit d'une grille 2D de pixels, de dimensions `(LARGEUR, HAUTEUR)`. Chaque pixel à l'écran est repéré par ses coordonnées `(x, y)`.

Attention. L'origine du repère est le coin supérieur gauche de la fenêtre. Les axes du repère de la fenêtre graphique sont orientés vers la droite et **vers le bas**. Ainsi, le pixel `a` de l'image ci-contre a pour coordonnées `(0, 0)`, le pixel `b` a pour coordonnées `(5, 2)`.



Les couleurs sont gérées à l'aide d'une **palette**. La palette par défaut est celle indiquée dans la documentation. Cette palette définit pour chaque couleur un **code**, qui permet de l'identifier.



Ainsi, pour faire référence à la couleur orange (dont la valeur RGB est `(211, 132, 65)`), on utilisera le nombre 9.

Enfin la bibliothèque met à disposition différentes fonctions permettant d'afficher des formes dans la fenêtre, de gérer les touches du clavier, etc. À l'aide de ces fonctions, on peut écrire un premier programme très simple :

Code python : premier exemple

```python
import pyxel

pyxel.init(128, 128, title="Mon premier programme Pyxel")

pyxel.cls(0)

x = 10
y = 10
pyxel.text(x, y, "Mon premier programme avec", 7)  # Blanc (7)
pyxel.text(x, y + 10, "Pyxel!", 8)  # Rouge (8)

pyxel.show()
```

Fonction	**Description**
`init(width, height, [title], [fps])`	Initialise l'application Pyxel avec un écran de taille `(width, height)`. Il est possible de passer comme options : le titre de la fenêtre avec `title`, le nombre d'images par seconde avec `fps`,
`cls(col)`	Efface l'écran avec la couleur `col`.
`text(x, y, s, col)`	Dessine une chaîne de caractères `s` de couleur `col` à `(x, y)`.
`show()`	Affiche l'écran jusqu'à ce que la touche `Esc` soit appuyée.

Remarque. Dans la documentation, certains paramètres de certaines fonctions sont écrits entre crochets. C'est le cas par exemple pour la fonction `init` pour le paramètre `title`, ou encore `fps`. Cela veut dire qu'il s'agit de paramètres **optionnels**. On peut les utiliser, ou pas, lorsque l'on appelle la fonction en question. Si on les utilise, on doit préciser leur nom.
Par exemple, on peut écrire :
`pixel.init(128, 128)` ou bien `pixel.init(128, 128, title = "Mon programme")`
Dans le premier cas, la fenêtre créée n'aura pas de titre particulier, dans le second cas, le titre de la fenêtre sera `"Mon premier programme"`.

Question 1. En vous inspirant du premier premier exemple, écrire un code python qui permette de réaliser l'affichage suivant.
La fenêtre est de taille 128 par 128. La couleur du fond de l'image est la couleur noire (0). Toutes les lettres peuvent être inscrites dans un carré de 4×4 pixels.

- Le texte `"Pyxel c'est magique"` est écrit en bleu (1) et `"<3"` en rouge (8).
- Un caractère sur deux de la chaine `"Vive le binaire"` est en blanc (7), l'autre est en vert (11).
- Chaque caractère de la chaîne de caractère `"Et toutes les couleurs (...)"` est de couleur différente : d'abord en bleu (1), puis en violet (2) etc jusqu'à beige (15) après quoi on recommence avec la couleur bleu (1), puis violet (2) etc.
- Le lion est généré de manière ASCII à l'aide de la chaîne de caractère `txt_lion` (le caractère spécial `"\n"` indique un retour à la ligne) en orange (9).

Résultat

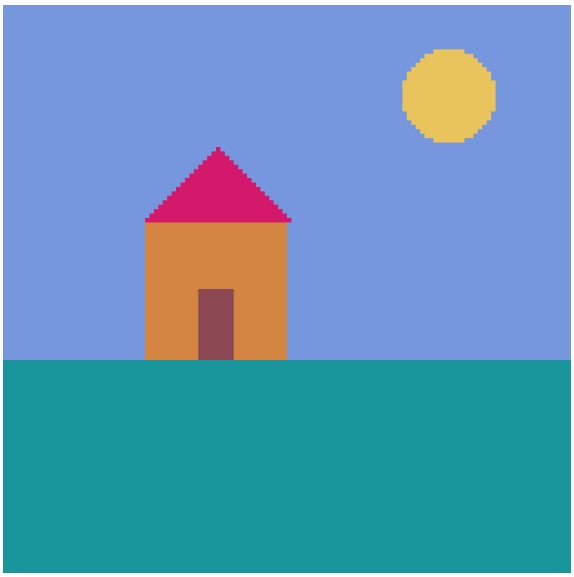
```
txt_lion = """
              ,%%%%%%%%,
            ,%%/\%%%%/\%%
           ,%%%\c "" J/%%%
 %.       %%%%/ o   o \%%%
 `%%.     %%%%     _   |%%%
  `%%     `%%%%(__Y__)%%'
  //       ;%%%%`\-/%%%'
 ((       /   `%%%%%%%'
  \\     .'          |
   \\  /       \  | |
    \\/         ) | |
     \         /_ | |__
     (___________)))))))
"""
```

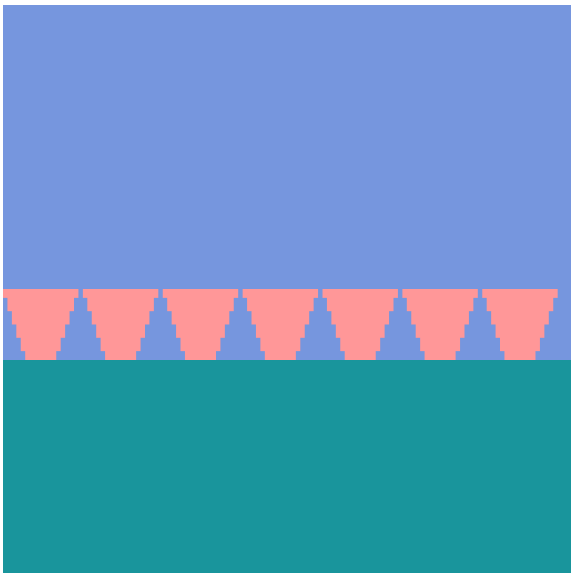
2 Fonctionnalités graphiques

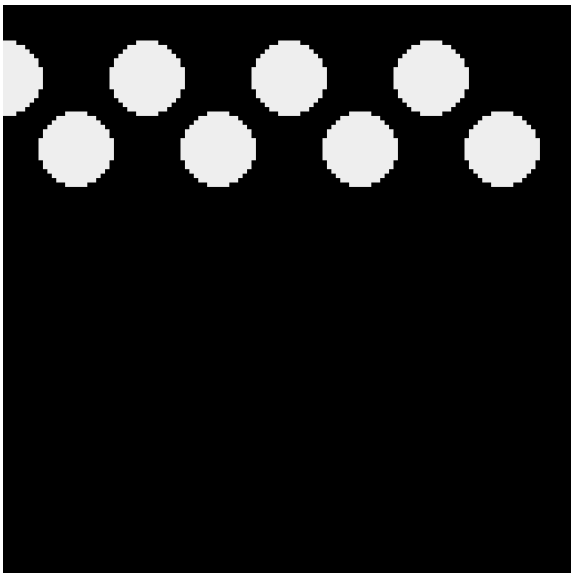
La bibliothèque `pyxel` dispose également des fonctions suivantes.

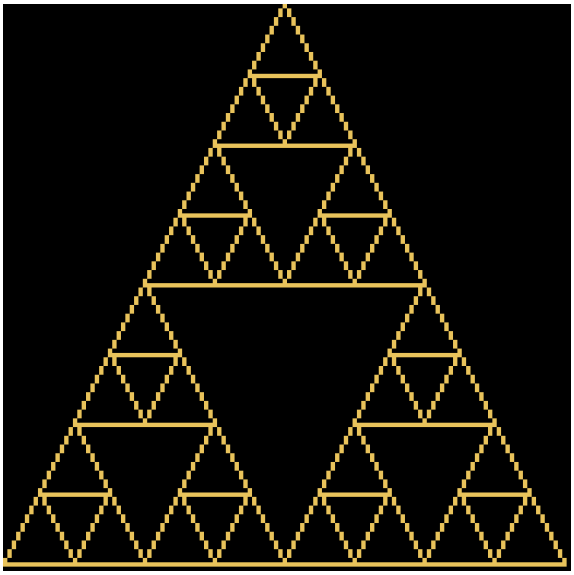
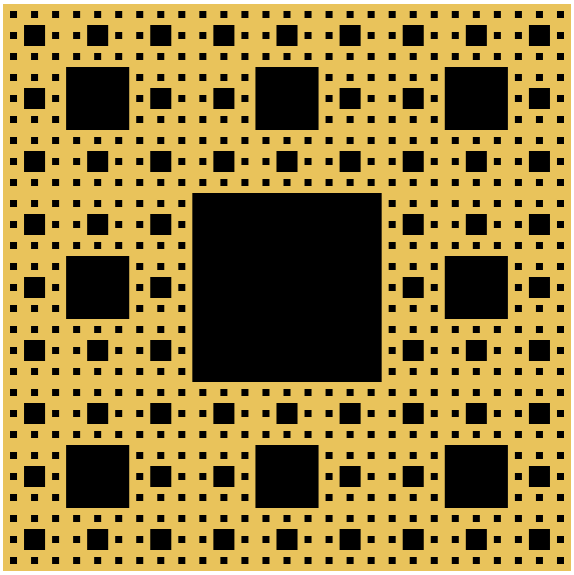
Fonction	Description
`pget(x, y)`	Renvoie la couleur au pixel `(x, y)`.
`pset(x, y, col)`	Dessine un pixel de couleur `col` à `(x, y)`.
`line(x1, y1, x2, y2, col)`	Dessine une ligne de couleur `col` de `(x1, y1)` à `(x2, y2)`.
`rect(x, y, w, h, col)`	Dessine un rectangle de largeur `w`, de hauteur `h` et de couleur `col` à partir de `(x, y)`.
`rectb(x, y, w, h, col)`	Dessine les contours d'un rectangle de largeur `w`, de hauteur `h` et de couleur `col` à partir de `(x, y)`.
`circ(x, y, r, col)`	Dessine un cercle de rayon `r` et de couleur `col` à `(x , y)`.
`circb(x, y, r, col)`	Dessine le contour d'un cercle de rayon `r` et de couleur `col` à `(x, y)`.
`elli(x, y, w, h, col)`	Dessinez une ellipse de largeur `w`, de hauteur `h` et de couleur `col` à partir de `(x, y)`.
`ellib(x, y, w, h, col)`	Dessinez le contour d'une ellipse de largeur `w`, de hauteur `h` et de couleur `col` à partir de `(x, y)`.
`tri(x1, y1, x2, y2, x3, y3, col)`	Dessine un triangle avec les sommets `(x1, y1)`, `(x2, y2)`, `(x3, y3)` et de couleur `col`.
`trib(x1, y1, x2, y2, x3, y3, col)`	Dessine les contours d'un triangle avec les sommets `(x1, y1)`, `(x2, y2)`, `(x3, y3)` et de couleur `col`
`fill(x, y, col)`	Remplit la zone connectée avec la même couleur que `(x, y)` avec la couleur `col`.

Question 2. À l'aide des fonctions ci-dessus, reproduire les dessins suivants.







Question 3. Laisser libre cours à votre imagination pour réaliser un joli dessin.

Question 4. Challenge (niveau terminale NSI).
Il est possible de réaliser des dessins de manière récursive.